Technical debt register — 2026-10-20

A measurement-first survey of the whole monorepo, run after the pre-share audit. **Nothing here has been changed.** Every item carries its evidence, a size, a risk, and a recommended verdict — the verdict is the founder's to set. Supersedes the open items of `architecture-survey-2026-07-20.md` (the Tier 1 money items there were fixed in #301–#303).

Verdicts: **FIX NOW** · **FIX WHEN TOUCHED** · **LEAVE** (with the reason recorded, so it is a decision and not an oversight).

Sizes: S < 1h · M < 1 day · L > 1 day. "First issue" marks work a developer can do while learning the codebase.

The shape of the thing

Source files (apps/web, excl. tests/stories)	346
Lines	55,187
Comment lines	8,854 (16%)
Dated founder rulings in comments	549
TODO / FIXME / HACK	0
Routes	30
Component files	211 (46 with a story = 22%)
Tests	1,458 web + 177 admin, 0 failing
`as unknown as`	29 (22 in money/auth paths)
`: any`	10 · `@ts-ignore` 0 · `eslint-disable` 68

The headline is the last two rows of the first column. There are no TODOs because the history lives in the comments instead: 549 dated rulings inside source files, which is where decisions go when a project has no decision log. That is the single biggest obstacle to a second person (or a second codebase) learning this one.

Tier 1 — correctness, security, trust

1.1 Three database views bypass row-level security · S · low risk · FIX NOW

The July survey recorded "no RLS anywhere". **That is now out of date and was corrected by measurement:** RLS is enabled on 34 of 38 relations on production, every policy is SELECT-only, and writes with the public key are refused (`insert into charities` → HTTP 401, "new row violates row-level security policy"). Pledges, users, gift-aid declarations and charity profiles all return zero rows to the public key.

The exception is three **views**, which run as their owner and so read straight past RLS on the tables beneath them:

View	Reads from	Public key can read it
`register_profile_floor`	`register_charities` (locked)	yes — verified, 1 row returned
`charity_profiles_stale`	`charity_profiles` + `register_charities` (both locked)	yes (times out on size, not denied)
`charity_profiles_unkeyable`	`charities`	yes

What leaks is public Charity Commission data plus `consent_status` and `is_active` for every charity — not money, not personal data. So: real, narrow, cheap. Two of the three (`register_profile_floor`, `charity_profiles_stale`) are referenced only by the migration that created them and by nothing in the application.

Fix: `alter view … set (security_invoker = on)` on all three, or `revoke all on … from anon, authenticated`. Drop the two unused ones.

1.2 The route that decides what a card is charged has no test · M · low risk · FIX NOW

`app/api/stripe/payment-intent/route.ts` computes `totalPence` from the pledge, tip and top-up parts and creates the PaymentIntent. It is the only money path in the app with no test file of its own; every other one has tests (`webhooks/stripe`, `webhooks/clerk`, both crons, all three action files, `stripe-verify`, `card-fee`, `disbursement/split`, `charity-consent`). The cases worth pinning: amount is server-computed and ignores a client-supplied total, each part is capped, `totalPence <= 0` is refused, metadata binds the favpoll and poll, and the consent gate refuses under `consent-first`.

`lib/poll-standings.ts` is the other untested module that produces numbers users see.

1.3 No server action validates its input · M · medium risk · FIX NOW

Across the six server-action files there are **zero** zod schemas or `safeParse` calls. Every action reads `formData.get(...) as string` and trusts the shape. Auth itself is handled (`[id]/actions.ts`: 13 exports, 9 `auth()` calls, 12 `created_by` checks; `new/actions.ts`: 4 of 4), so this is about malformed input, not impersonation. A shared `parse(schema, formData)` helper plus one schema per action is the contained fix.

`app/pledges/withdraw/actions.ts` deliberately has no `auth()` — it is authorised by `guest_token` alone, which is correct for a link emailed to a guest. Worth a comment saying so, because it reads like an omission.

1.4 Actions still throw where the project decided they should return · M · low risk · FIX WHEN TOUCHED

`lib/action-result.ts` exists and is tested, and the manage surface was converted in #1049, but **none of the six action files use it** (`asResult` / `ActionResult` appear 0 times in all six). Next redacts

thrown messages in production, so a failure in the wizard or the pledge path reaches the user as a generic error. Convert one file per PR, starting with the pledge path.

Tier 2 — structure (what makes this codebase hard to learn)

2.1 549 dated rulings live in source comments, not in a decision log · L · low risk · FIX NOW (process)

Examples sit in nearly every file: "(founder, 2026-09-16, after a tap-advance audition)", "THREE TO A ROW SINCE 2026-08-27, not two". They are genuinely valuable — they are the reasons — but they are invisible to anyone reading the repo top-down, they bloat the files they live in, and they cannot be searched by decision. Moving the load-bearing ones into ADRs under `docs/adr/` and leaving a one-line pointer in the code is the single highest-value change for a second reader, and it is the prerequisite for the Matt Pocock skills (`to-prd`, `to-issues`, `triage` all assume a tracker and a doc layout).

2.2 Seven files over 700 lines mix layout, data, animation and copy · L each · medium risk · FIX WHEN TOUCHED

File	Lines	What it holds
`lib/pairing-table.ts`	1,374	data table — fine as data, but it is code
`components/landing/hero.tsx`	1,073	layout + beat state machine + copy + carousel
`app/favpolls/[id]/manage/manage-client.tsx`	1,014	seven sections in one client component
`lib/story-engine.ts`	956	generation rules
`app/favpolls/[id]/actions.ts`	878	13 exported actions, three pledge paths
`components/new-favpoll-wizard/use-wizard-state.ts`	832	whole wizard state
`components/hero-demo-panel/demo-card.tsx`	819	demo rendering + the still beat

The seams are visible in each: `hero.tsx` splits into layout / `useStillBeat` / copy; `manage-client.tsx` splits by section; `actions.ts` splits into pledge-by-card, pledge-from-pot and guest-pledge over one shared pipeline (the three duplicate their verification and insert logic today).

2.3 Two libraries are copy-pasted between web and admin, and the copies are formatted differently · M · low risk · FIX NOW

`lib/register-mirror.ts` (298 / 266 lines) and `lib/charity-commission.ts` (636 / 635) exist in both apps. Diffed raw they differ on 308 and 574 lines; diffed ignoring semicolons and trailing commas they differ on 66 and 43 — **the two apps use different Prettier settings**, so a real behavioural drift would be invisible in the formatting noise. This has already bitten once: memory records "copied verbatim into admin — keep the two in sync".

Fix: one Prettier config at the root, then move both modules into `packages/` and import them. `lib/utils.ts` is identical in both and can go the same way.

2.4 One dead UI subsystem and eleven dead components · S · low risk · FIX NOW · first issue

Verified by grep, zero production importers: `ui/field` (249), `ui/form` (152), `ui/ticker` (120), `ui/tooltip-icon-button` (43), `ui/occasion-tag` (19), `landing/honour-charity-love-venn` (219), `landing/guest-book-vignette` (91), `landing/goal-vignette` (60), `favpoll-card/section-label` (29), `pledge-dialog/step-indicator` (23), `keepsake/print-button` (20). About 1,025 lines.

Separately, the **`components/pledge-card/` subsystem is dead except `pledge-breakdown.tsx`**: `index`, `preview-pledge-card`, `live-pledge-card`, `pledge-card-wrapper`, `amount-input` and `amount-presets` have no importer outside their own directory and their stories. The pledge dialog's own step components replaced them. Six more files.

Note the trap: five of these have Storybook stories, so "it has a story" is not evidence of use.

2.5 33 of 177 message keys are dead · S · low risk · FIX NOW · first issue

Including the whole `landing.how.*` set (10 keys), `celebrations.moment.*` and `celebrations.presence.*` (7), and `landing.invariant` — which the copy sweep edited yesterday for a line that no component renders. Counted with dynamic prefixes (`home.assure.`, `home.capability.`) accounted for.

2.6 Type escapes concentrate in the money and auth paths · M · medium risk · FIX WHEN TOUCHED

29 `as unknown as` in the web app, 22 of them under `app/api`, `app/favpolls`, `lib/stripe-verify`, `lib/disbursement` or `lib/supabase`. The pattern is `(row.favpoll_polls as unknown as { favpoll_id: string })` — working around Supabase's generated join types. One typed helper for joined rows would remove most of them. Also 12 `process.env.X!` non-null assertions, which turn a missing variable into a runtime crash rather than a startup error.

Tier 3 — tooling and process

3.1 CI never builds either app · S · low risk · FIX NOW

Jobs today: Test, Typecheck, Lint, Format check, and E2E (advisory). There is **no `pnpm build` anywhere**. The repo's own `CLAUDE.md` records a trap that only a build catches — a `"use server"` module may only export async functions, and "typecheck and vitest miss it". Add a build job for both apps.

Also absent: a coverage report, a bundle-size check, a dependency audit, and a migration check. The component-tree check landed today (#1092).

3.2 Storybook is current but covers a fifth of the library · M–L · low risk · FIX WHEN TOUCHED

46 of 211 component files have a story; all 178 stories pass as browser tests in CI, so what exists is accurate. The gaps are the big surfaces:

Area	Files	Stories
`components/landing`	32	1
`components/ui`	33	12
`components/manage`	6	0
`components/keepsake`	6	0
`components/print-pack`	6	0
`components/room-shell`	3	0
`components/heroes`	2	0
`favpoll-sheet`, `guest-book`	2	0

3.3 Fifteen stories fail in a static Storybook build · S · low risk · FIX NOW

`FavpollListCard` (5), `PledgeDialog` (5) and `PollSection` (5) throw "invariant expected app router to be mounted" and "`__dirname` is not defined" when opened from `storybook build` output, while passing under vitest. Cause: `.storybook/main.ts` and `vitest.config.ts` keep **separate alias lists** that have drifted. Fix: one exported alias map imported by both. Until then Storybook cannot be deployed for anyone to look at.

3.4 No tracker, no ADRs, no templates · M · low risk · FIX NOW (process)

`references/` holds 90 markdown documents and `PROJECT.md` is 1,253 lines; together they are a decision log in all but name, but nothing is indexed, dated-but-superseded entries are not marked, and none of it is in the format the engineering skills expect. There is no `.github` issue or PR template, no CODEOWNERS, no CONTRIBUTING. `references/GLOSSARY.md` exists and is the natural seed for `CONTEXT.md`.

The minimal skeleton: run `/setup-matt-pocock-skills` once (it asks three questions — tracker, triage labels, doc layout), then turn this register into issues with `/to-issues`, then work them with `tdd`.

Recommended order

1. **1.1** views (an afternoon, closes the only security finding)
2. **2.4 + 2.5** dead code and dead keys (first issues, ~1,100 lines out)
3. **3.1** build in CI, **3.3** one alias map
4. **3.4** process skeleton, then **2.1** ADRs as you touch each area
5. **1.2 + 1.3** payment-intent tests and input validation
6. **2.3** shared packages, **1.4** action results, **2.2** splits — as you touch them

A note on the React Native plan

Nothing in `apps/web` is reusable by a native client as it stands: the UI is Tailwind on the DOM, and the data layer is Next server actions, which a native app cannot call. A native client needs an HTTP API that does not exist yet — today there are 10 route handlers, and they are webhooks, crons and two entitlement-gated reads. Building that API is a real project and the web app would benefit from it too (it would give the pledge pipeline in 2.2 a single front door). It is a reasonable goal; it is not a cheap way to learn the codebase. This register plus the ADRs in 2.1 is the cheaper way.